트램 전방 충돌 위험 알림 시스템 상세 보고서

동아리 트램 캡스톤 · 2026년 10월 · 발표 자료 「트램_발표3_최종.pptx」의 상세판

트램 앞에 단 카메라 하나로 선로 위 사람과 차량을 찾고, 실제 거리(m)와 충돌까지 남은 시간(TTC)을 계산해 안전·주의·경고·위험 4단계로 알리는 장치를 1:10 모형에서 구현합니다. 라즈베리파이 5가 촬영부터 추적·판단·경보까지 맡는 '판단 주체'이고, 노트북은 가장 무거운 물체 검출과 선로 검출만 돕습니다. 노트북과 연결이 끊기면 파이는 즉시 칼만 필터 예측으로 판정을 이어 가고, 1초가 넘으면 파이 안의 경량 YOLO로 직접 검출합니다. 이 1초 기준은 3,000회 시뮬레이션으로 정했습니다.

이 보고서가 답하는 질문

질문	짧은 답	자세히
카메라 하나로 재는 거리는 상대적이지 않나?	길이를 아는 레일 간격(1.435 m)을 자로 쓰고 카메라를 보정하므로 실제 m 단위 절대 거리입니다.	4.4절
칼만 필터는 어떤 것을 쓰나?	측정을 먼저 미터로 바꾸므로 선형 칼만 필터 + 등속도 모델이면 충분합니다. EKF·UKF는 필요 없습니다.	4.5절
모델 학습은 어떻게 하나?	COCO 사전학습 YOLO11n을 먼저 재 보고, 재현율이 부족하면 시험장 사진 300~500장으로 미세조정합니다.	6장
예측 모드는 쓸모 있나? 파이 자체 검출은 꼭 필요한가?	예측은 약 1초까지 유효하고, 그 뒤에는 헛경보가 급증하며 새 물체를 못 봅니다. 그래서 둘 다 필요합니다.	5.2~5.3절
활용할 코드나 예제가 있나?	직접 만든 예제 11개(00~10)가 있고, 07~10은 이번에 추가해 PC에서 실행을 확인했습니다.	7장

발표 2 이후 바뀐 점

- 연결 끊김 대응을 3단계(0.3초 유지 → 1.5초까지 예측 → 자체 검출)에서 **2단계(즉시 예측 → 1초부터 자체 검출)**로 바꿨습니다. 시뮬레이션에서 '마지막 값 유지'는 0.3초만 지나도 통로 안 사람의 19%를 놓쳤기 때문입니다.
- 예측 모드에서 칼만 필터의 불확실성(2σ)만큼 위험 통로를 넓혀 판정하도록 했습니다. 이렇게 하면 3초까지 놓침이 0.1% 이하였습니다.
- 파이의 자체 검출 모델은 평소에도 메모리에 올려 두어 전환 시간을 없앱니다.
- 칼만 추적기(07), 끊김 시뮬레이션(08), 노트북 서버(09), 파이 클라이언트(10) 예제를 새로 만들었습니다.

1. 개요

1.1 문제

트램은 레일 위를 달리기 때문에 장애물을 피해 갈 수 없고, 제동거리가 깁니다(50 km/h에서 약 94 m, 추정치). 도심 선로 옆에는 보행자가 많아서 '사람이 보인다'만으로 경보하면 경보가 계속 울려 운전자가 무시하게 됩니다. 그래서 **선로 통로 안에 있거나 들어오는지, 몇 초 뒤 부딪히는지**를 계산해 단계별로 알려야 합니다.

1.2 목표와 범위

- 카메라 하나로 선로 위 사람·자전거·차량 검출
- 실제 거리(m)와 접근 속도, TTC, 필요 감속도 계산
- 안전·주의·경고·위험 4단계 경보 (부저·LED·화면)
- 시험은 하드보드로 만든 1:10 모형 선로(궤간 14.35 cm)와 피규어로 진행

1.3 최소 성능 목표 (제안)

지표	목표	측정 방법
처리 속도	10 FPS 이상	파이 로그의 프레임 간격
지연	촬영 → 경보 0.3초 이하	화면 시계를 촬영해 경보 시각과 비교
감지 거리	모형 5 m (실제 50 m 상당)	선로판 눈금
놓침	위험 시나리오에서 0건	시나리오 S1·S3 각 10회
오경보	10% 이하	시나리오 S2

2. 사용 부품

보드(라즈베리파이 5 8GB)는 이미 있으므로 제외하고, 예산 20~30만원 안에서 구성했습니다. **가격은 기억에 기반한 추정치이므로 발주 전에 국내 판매가를 확인해야 합니다.**

품목	쓰는 곳	예산(원)
라즈베리파이 카메라 모듈 3 + 파이 5용 케이블	전방 촬영 (수평 화각 약 66°, HDR)	50,000
공식 27W 전원 · 액티브 쿨러 · 케이스	안정 동작, 발열 대비 (자체 검출 시 부하 큼)	40,000
microSD 64 GB (A2)	OS · 로그	15,000
5 GHz 공유기	파이 ↔ 노트북 전용 무선망	30,000
보조배터리 (USB-C PD, 5V 3A 이상)	주행체 위 파이 전원	40,000
부저 · LED 3개 · 점퍼선	소리 · 빛 경보	10,000
하드보드지 · 노란 테이프 · 피규어 · 눈금	카메라 거치대, 1:10 선로판	30,000
필수 합계		**215,000**
선택: 주행체 (RC카 + 엔코더)	일정 속도 주행, 속도 정답	80,000

이미 있는 장비: 라즈베리파이 5 (8GB), 노트북(Core Ultra 7 255H, RAM 32 GB, Intel Arc 140T, OpenVINO 가속 가능), 팀원 휴대폰(초기 영상 촬영).

배선 예시(예제 10의 기본값, 배선에 맞게 바꿈): 부저 GPIO17, LED GPIO22·23·24 (주의·경고·위험).

3. 각 장치별 기능과 연결

3.1 연결 그림

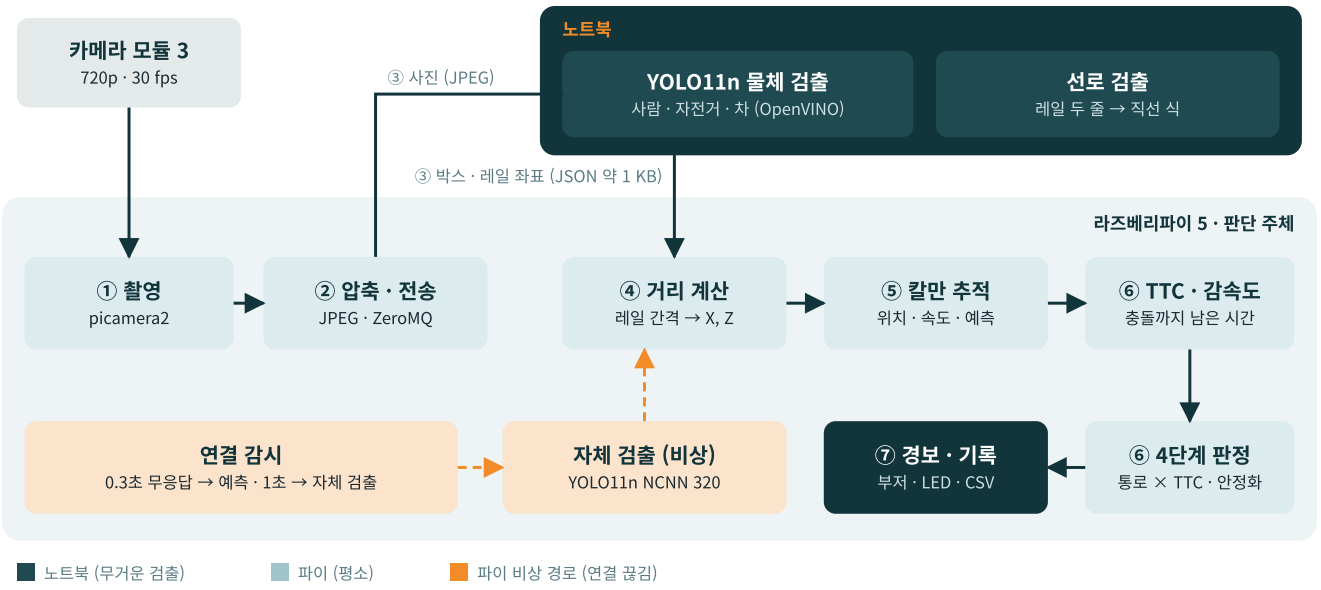


그림 1. 장치별 기능과 연결 (번호 = 처리 순서)

번호는 처리 순서입니다. 노트북에는 사진이 가고 숫자(박스 좌표와 레일 직선 식)만 돌아오므로, **판정에 필요한 상태(추적 중인 물체, 속도, 등급)는 모두 파이에 있습니다.** 그래서 노트북이 끊겨도 파이는 판정을 멈추지 않습니다.

3.2 장치별 역할

장치	하는 일	쓰는 기술	예제
카메라 모듈 3	720p 30 fps 촬영, 차량 중심선 위 고정	picamera2	10
라즈베리파이 5	촬영·JPEG 압축·전송	OpenCV, ZeroMQ	10
라즈베리파이 5	거리 계산, 칼만 추적, TTC·필요 감속도, 4단계 판정, 경보, 기록	numpy, gpiozero	04, 05, 07, 10
라즈베리파이 5	연결 감시, 예측 모드, 자체 검출(비상)	칼만 예측, YOLO11n NCNN 320	07, 10
노트북	YOLO11n 물체 검출, 선로 검출, 화면 표시	Ultralytics + OpenVINO, OpenCV	02, 03, 09
하드보드 모형	1:10 선로판(궤간 14.35 cm), 50 cm 눈금, 카메라 거치대	–	–

3.3 주고받는 메시지

파이 → 노트북: `[헤더 JSON, JPEG 바이트]`, 헤더는 `{"frame_id": 812, "t_capture": 81.2}`

노트북 → 파이 (약 1 KB):

```
{"frame_id": 812, "t_capture": 81.2, "proc_ms": 23.5,
 "dets": [{"name": "person", "conf": 0.87, "box": [612, 301, 640, 372]}],
 "rails": {"left": [-0.71, 905.2], "right": [0.69, 389.0], "y_top": 396, "y_bottom": 719}}
```

`t_capture`로 결과가 몇 초 전 사진의 것인지 알 수 있으므로, 0.3초보다 오래된 결과는 버립니다.

3.4 지연 예산 (추정, 2주차 실측)

구간	예상
촬영 + JPEG 압축 (파이)	약 15~30 ms
무선 전송 왕복 (5 GHz 전용망)	약 20~80 ms
YOLO11n 검출 (노트북, OpenVINO)	약 15~40 ms
거리·칼만·판정·경보 (파이)	1 ms 미만
합계	**약 0.1~0.2초 (목표 0.3초 이하)**

같은 PC 안에서 예제 09·10을 돌렸을 때 왕복은 약 10~15 ms였습니다(무선 지연 미포함).

4. 사용 알고리즘

4.1 처리 흐름 한눈에

단계	입력 → 출력	방법	장치	예제
① 촬영 · 전송	카메라 → JPEG 사진	picamera2, ZeroMQ	파이	10
② 선로 검출	사진 → 레일 두 줄의 직선 식	노란 테이프 색 분리 + 직선 맞춤	노트북 (끊기면 파이)	03
③ 물체 검출	사진 → 사람·차 박스	YOLO11n	노트북 (끊기면 파이)	02, 09
④ 거리	박스 발끝 + 레일 간격 → X, Z (m)	궤간 비례 + 사람 키 보조	파이	04
⑤ 추적 · 예측	X, Z → 위치·속도·불확실성	선형 칼만 필터 + 게이트 매칭	파이	07
⑥ 판단	위치·속도 → TTC, 필요 감속도, 등급	규칙표 + 깜빡임 방지	파이	05
⑦ 경보 · 기록	등급 → 부저·LED·CSV	gpiozero	파이	10

4.2 선로 검출

모형에서는 레일을 노란 테이프로 만들어 HSV 색 범위로 분리합니다. 화면 아래쪽 55%를 25줄로 나눠 줄마다 노란 픽셀 묶음 사이의 가장 큰 빈칸을 기준으로 왼쪽·오른쪽 레일 점을 찾고, 각 레일을 직선(곡선 구간은 2차식)으로 맞춥니다. 결과는 "화면 높이 y에서 레일이 어디 있는가"를 알려 주는 식이며, 이 식으로 그 높이의 레일 간격(픽셀)과 위험 통로(차체 폭/2 + 여유 0.5 m)를 계산합니다. 학습이 필요 없고 계산이 가벼워 파이에서도 돌아갑니다. 실제 선로 영상으로 넓힐 때는 RailSem19로 학습한 분할 모델(YOLO11n-seg)로 바꿉니다.

4.3 물체 검출

Ultralytics YOLO11n(가장 작은 모델)을 씁니다. COCO로 사전학습되어 사람(0), 자전거(1), 차(2), 오토바이(3), 버스(5), 트럭(7)을 바로 찾을 수 있습니다. 노트북에서는 OpenVINO로 변환해 Intel GPU/NPU로 가속하고, 파이 비상용은 NCNN 형식, 입력 320 px로 줄여 씁니다. 학습 계획은 6장에 있습니다.

4.4 거리 측정: 왜 '절대' 거리인가

카메라 하나로 보이는 크기만 쓰면 "가까워졌다 / 멀어졌다"만 알 수 있어 상대적입니다. 하지만 **길이를 이미 아는 물체가 같은 높이(같은 거리)에 있으면** 화면의 픽셀을 실제 미터로 바꿀 수 있습니다. 선로에서는 레일 간격(표준 궤간 1.435 m, 모형 14.35 cm)이 화면 어디서나 그 자리의 '자' 역할을 합니다.

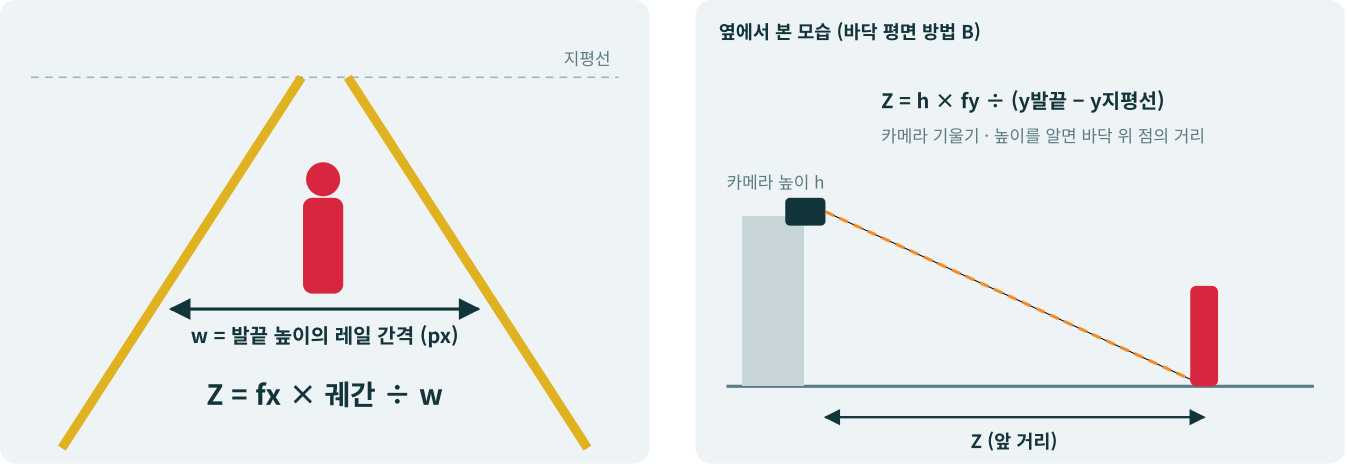


그림 2. 레일 간격으로 재는 절대 거리 (왼쪽: 카메라 화면, 오른쪽: 옆에서 본 바닥 평면 방법)

방법 A (주): 레일 간격. 사람 박스의 발끝 높이 y에서 레일 간격이 w 픽셀이면 앞 거리 Z = fx × 궤간 ÷ w 입니다. 옆 거리 X는 같은 높이에서 (발끝 x − 선로 중심 x) × 궤간 ÷ w 로 구합니다. fx(초점거리, 픽셀 단위)는 카메라 보정으로 한 번 구합니다.

방법 B (보조): 바닥 평면. 카메라 높이 h와 기울기를 알면, 바닥에 닿은 점의 화면 높이로 거리를 구할 수 있습니다. 레일이 가려졌을 때 씁니다.

방법 C (보조): 물체 크기. 사람 키 1.7 m(모형 17 cm)를 가정해 Z = fy × 키 ÷ 박스 높이. 키가 사람마다 달라 오차가 크므로 마지막 수단입니다.

세 값은 신뢰도 순(A → B → C)으로 고르고, 시간에 따른 흔들림은 칼만 필터가 다시 줄입니다.

정확도. 카메라 모듈 3(수평 화각 약 66°, 사양 확인 필요)을 1280 px 폭으로 쓰면 fx ≈ 640 ÷ tan 33° ≈ 985 px입니다.

모형 거리	레일 간격	1 px 틀리면	실제 환산
1 m	141 px	±0.7 cm (0.7%)	10 m
2 m	71 px	±2.8 cm (1.4%)	20 m
3 m	47 px	±6.4 cm (2.1%)	30 m
5 m	28 px	±18 cm (3.5%)	50 m

거리 오차는 거리의 제곱에 비례해 커집니다(ΔZ = Z^2 ÷ (fx × 궤간) × Δw). 레일은 25줄의 점으로 직선을 맞추므로 실제 간격 오차는 1 px보다 작아질 수 있습니다.

카메라 보정 절차. A4 체커보드를 여러 각도로 20장 찍고 OpenCV `cv2.calibrateCamera`로 fx, fy와 렌즈 왜곡 계수를 구합니다. 왜곡을 보정한 뒤 거리를 계산하면 화면 가장자리 오차가 줄어듭니다. 결과는 `config.py`의 `HFOV_DEG`(또는 fx 값)에 넣습니다.

검증. 선로판에 50 cm마다 눈금을 붙이고 피규어를 각 눈금에 놓아 측정값과 정답을 비교합니다. 거리 구간별 평균 오차와 최대 오차를 표로 보고하며, 목표는 모형 3 m 안에서 오차 5% 이하(제안)입니다. 확장안으로는 파이 5의 두 번째 카메라 단자에 카메라를 하나 더 달아 스테레오로 비교할 수 있습니다.

4.5 칼만 필터: 어떤 것을, 왜

칼만 필터는 "예측 → 보정"을 반복하는 추정 방법입니다. 지난 위치와 속도로 다음 위치를 먼저 예측하고, 새 측정이 오면 예측과 측정 중 더 믿을 만한 쪽에 무게를 두어 섞습니다. 측정이 오지 않으면 예측만 하면 되므로 연결 끊김 대응에 그대로 쓸 수 있습니다.

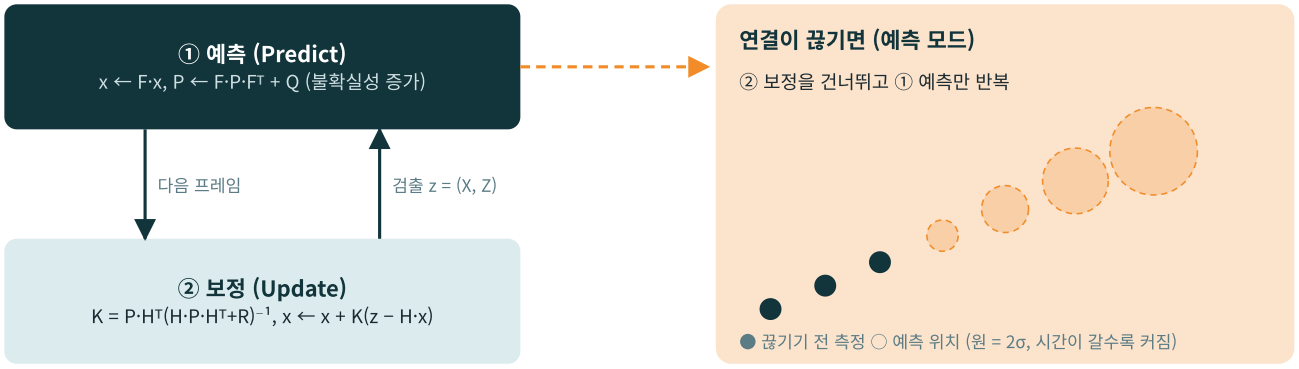


그림 3. 칼만 필터의 예측-보정 반복과 연결이 끊겼을 때의 예측

종류 비교와 선택

종류	언제 쓰나	계산량	우리 판단
선형 KF	상태와 측정이 선형 관계	매우 작음	**선택.** 측정을 미리 미터로 바꾸므로 선형
EKF (확장)	픽셀 좌표를 바로 측정으로 쓸 때 (비선형)	작음	불필요
UKF (무향)	비선형이 강할 때	중간	불필요
IMM	걷기·멈춤 등 여러 움직임 모델을 섞을 때	모델 수만큼	확장 목표 (급정지 보행자 대응)
파티클 필터	분포가 여러 봉우리일 때	큼	불필요

SORT 논문 방식처럼 화면 박스 [u, v, s, r]에 칼만 필터를 거는 방법도 있습니다. 이것은 노트북 쪽 ByteTrack 안에 이미 들어 있으므로, 파이는 **바닥 평면 미터 좌표**에서 추적합니다. 그래야 예측 결과를 TTC와 통로 안/밖 판단에 곧바로 쓸 수 있습니다.

설계 값 (예제 07)

항목	값	뜻
상태 x	[X, Z, Vx, Vz]	옆·앞 위치(m)와 상대 속도(m/s)
전이 F	X ← X + Vx·Δt, Z ← Z + Vz·Δt	등속도 가정
측정 H	[I 0]	위치 X, Z만 측정
과정 잡음 Q	가속도 표준편차 1 m/s²	급정지·방향 전환 여유
측정 잡음 R	σZ = Z² ÷ (fx × 궤간) × 1.5 px, σX = 0.05 + Z ÷ fx × 3 px	멀수록 측정을 덜 믿음
짝 맞추기	마할라노비스 거리² < 9.21	새 검출이 어느 물체 것인지 (99% 범위)
트랙 삭제	1.5초 동안 측정 없음	사라진 물체 정리

실행 결과 (예제 07, 실제 크기, 트램 30 km/h, 사람이 1.4 m/s로 선로 쪽으로 이동, 10 FPS):

```
앞 거리 오차 평균  측정값 0.99 m  ->  칼만 0.51 m
마지막 추정: Z 19.1 m, 다가오는 속도 8.5 m/s (정답 8.3), 옆 속도 -1.49 m/s (정답 -1.40)
predict 1회 11 µs (이 PC 기준)
```

거리 오차가 절반으로 줄었고, 속도도 정답에 가깝게 추정됐습니다. 예측 한 번이 PC에서 약 11 µs이므로, 파이 5가 몇 배 느리더라도 물체 10개를 추적하는 데 1 ms가 안 걸립니다(파이 실측은 보드 도착 후).

4.6 TTC, 필요 감속도, 4단계 판정

- 접근 속도 v: 칼만 필터의 상대 속도(트램 속도 + 물체 속도)
- TTC = Z ÷ v (Lee, 1976의 '충돌까지 남은 시간' 개념)
- 필요 감속도 $a_{req} = v^2 \div (2 \times (Z - v \times \text{반응시간 1초}))$. 상용 제동 1.2 m/s²(예시값)보다 크면 일반 제동으로는 못 멈춘다는 뜻
- 진입 예상 시간 = 통로 경계까지 옆 거리 ÷ 선로 쪽 옆 속도

등급	조건
위험	통로 안이고 (TTC ≤ 3초 또는 필요 감속도 ≥ 1.2 m/s²)
경고	통로 안이고 TTC ≤ 5초, 또는 통로로 들어오는 중이고 진입 예상 시간 ≤ TTC ≤ 5초
주의	통로 안이지만 멀다, 또는 통로 바깥 1 m 완충 구역 안
안전	그 외

등급이 깜빡이지 않도록 **올릴 때는 2프레임 연속**이면 바로 올리고, **내릴 때는 1초 동안 계속 낮아야** 내립니다(예제 05의 LevelSmoother). TTC 5초·3초와 1.2 m/s²는 제안 초기값이며 시험으로 조정합니다.

4.7 참고 논문과 자료

아래 서지 정보는 기억에 기반해 정리했으므로, 발표 전에 제목·연도·학회를 원문으로 확인해야 합니다.

기술	참고 문헌	쓰는 곳
YOLO	Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016 · Ultralytics YOLO11 문서	물체 검출
칼만 필터	R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems", 1960	추적 · 끊김 시 예측
SORT	Bewley et al., "Simple Online and Realtime Tracking", ICIP 2016	추적 구조 (예측 → 매칭 → 보정)
ByteTrack	Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box", ECCV 2022	노트북 쪽 화면 추적
TTC	D. N. Lee, "A theory of visual control of braking based on information about time-to-collision", Perception 1976	충돌까지 남은 시간
RailSem19	Zendel et al., "RailSem19: A Dataset for Semantic Rail Scene Understanding", CVPR Workshops 2019	실제 레일 분할 학습 (향후)
RailSafeNet	오픈소스 (MIT), 트램 레일 분할 + 궤간 기반 거리 구역	옆 거리 구역 아이디어

우리가 새로 설계하는 부분은 ① 통로 × TTC × 필요 감속도를 묶은 등급 규칙, ② 연결 끊김 시 예측 → 자체 검출 전환과 그 기준을 정한 시뮬레이션, ③ 파이 중심의 역할 분담입니다.

5. 발생할 문제와 해결법

5.1 노트북과의 연결이 끊길 때

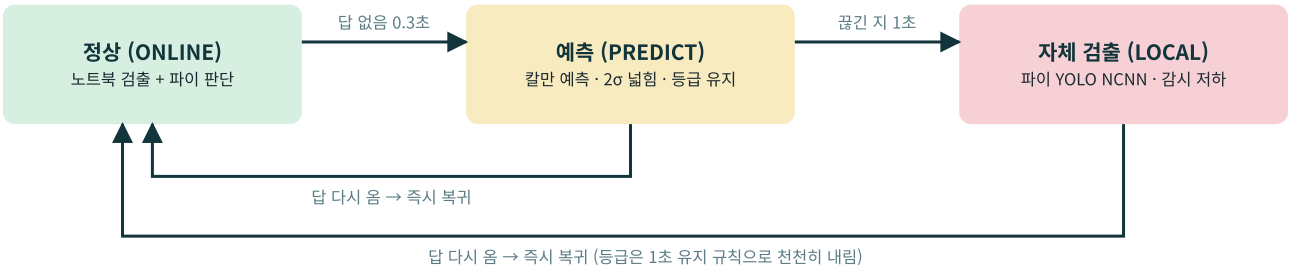


그림 4. 연결 상태에 따른 파이의 동작 모드

모드	들어가는 조건	파이가 하는 일	등급 규칙
정상	0.3초 안에 답이 옴	노트북 결과로 칼만 보정	일반 규칙
예측	답이 0.3초 안에 안 옴 (끊긴 뒤 0~1초)	칼만 예측만, 위치를 2σ만큼 선로 쪽·가까운 쪽으로 보수적으로 가정	끊기기 직전 등급보다 내리지 않음
자체 검출	끊긴 지 1초 이상	파이가 선로 검출 + YOLO11n NCNN 320 직접 실행, '감시 저하' 표시	최소 '주의'

답이 다시 오면 바로 정상 모드로 돌아갑니다. 통신은 ZeroMQ REQ/REP이며, 시간 안에 답이 없으면 소켓을 새로 만들어 다시 연결합니다(Lazy Pirate 패턴, 예제 10의 `Link`). 자체 검출 모델은 시작할 때 미리 메모리에 올려 두므로, 전환할 때 모델 로딩(수 초) 지연이 없습니다.

PC에서 시험한 결과 (예제 09 + 10, 합성 영상, 3초부터 2.5초 동안 전송 차단):

```
t=  2.7s  정상                  추적 1개  등급 경고  가장 위험한 물체  36.7 m  왕복 9 ms
t=  3.0s  예측                  추적 1개  등급 경고  가장 위험한 물체  33.1 m
t=  3.3s  예측                  추적 1개  등급 위험  가장 위험한 물체  30.3 m
t=  4.0s  자체 검출(감시 저하)  추적 1개  등급 위험  가장 위험한 물체  25.3 m
t=  5.3s  자체 검출(감시 저하)  추적 1개  등급 주의  가장 위험한 물체  15.2 m
t=  5.7s  정상                  추적 1개  등급 주의  가장 위험한 물체  12.7 m  왕복 9 ms
```

끊긴 동안에도 같은 물체를 계속 추적하며 등급을 올렸고, 연결이 돌아오자 정상으로 복귀했습니다.

5.2 예측 모드는 실제로 쓸모 있나 (시뮬레이션)

설정 (예제 08). 실제 크기 기준으로 트램 20~40 km/h, 보행자는 앞 30~60 m, 옆 3~7 m에서 0.8~1.8 m/s로 선로 쪽으로 걷습니다. 보행자의 50%는 계속 걷고, 25%는 갑자기 멈추고, 25%는 되돌아갑니다(가속도 1.5~3 m/s^2, 끊김 전후 아무 때나). 10 FPS로 2초 동안 측정(거리에 따른 잡음 포함)하다가 연결을 끊고, 끊긴 뒤 시점마다 세 가지 방법을 비교했습니다. 3,000회 반복했습니다.

- 마지막 값 유지: 끊기기 직전 측정값을 그대로 씀
- 칼만 예측: 칼만 필터의 예측 위치로 판정
- 칼만 + 2σ (우리 안): 예측 위치에서 불확실성 2σ만큼 선로 쪽으로 당겨 판정

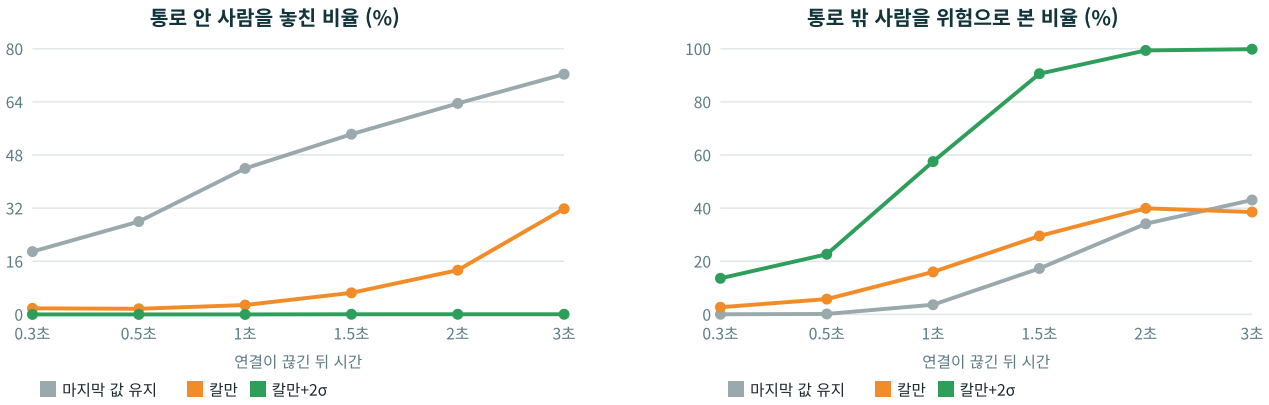


그림 5. 연결 끊김 시뮬레이션 3,000회 결과 (예제 08)

끊긴 뒤	앞 거리 오차 중앙값 (유지 / 칼만)	옆 거리 오차 중앙값 (유지 / 칼만)	통로 안 사람 놓침 (유지 / 칼만 / 칼만+2σ)	헛경보 (유지 / 칼만 / 칼만+2σ)
0.3초	2.58 / 0.30 m	0.37 / 0.07 m	18.9 / 1.8 / 0.0%	0.1 / 2.7 / 13.6%
0.5초	4.24 / 0.36 m	0.61 / 0.09 m	28.0 / 1.7 / 0.0%	0.2 / 5.8 / 22.7%
1초	8.39 / 0.50 m	1.17 / 0.15 m	44.0 / 2.8 / **0.0%**	3.7 / 16.0 / **57.5%**
1.5초	12.50 / 0.63 m	1.66 / 0.23 m	54.3 / 6.5 / 0.1%	17.3 / 29.5 / 90.6%
2초	16.64 / 0.78 m	2.09 / 0.34 m	63.5 / 13.3 / 0.1%	34.1 / 39.9 / 99.4%
3초	24.89 / 1.09 m	2.83 / 0.74 m	72.3 / 31.8 / 0.1%	43.1 / 38.5 / 99.8%

해석

1. **마지막 값 유지는 쓰면 안 됩니다.** 트램이 계속 다가오므로 0.3초만 지나도 앞 거리가 2.6 m 틀리고, 통로 안 사람의 19%를 놓칩니다. 그래서 발표 2의 '0.3초 유지' 단계를 없애고 끊기면 바로 예측합니다.
2. **칼만 예측은 위치를 잘 맞힙니다.** 1초 뒤에도 앞 거리 오차 중앙값 0.5 m, 옆 거리 0.15 m입니다. 다만 멈추거나 되돌아가는 사람 때문에 그냥 예측만 쓰면 1.5초에 6.5%를 놓칩니다.
3. **칼만 + 2σ는 놓침이 사실상 0이지만, 시간이 갈수록 '모두 위험'이 됩니다.** 불확실성 범위가 넓어져 1초에 통로 밖 사람의 58%, 1.5초에 91%를 위험으로 봅니다. 이것은 판단이라기보다 '모르니까 경보'에 가깝습니다.
4. **결론:** 예측 모드는 끊긴 직후 약 1초의 공백을 안전하게 메우는 데 유효하고, 그 뒤에는 **파이가 직접 보는 것 말고는 방법이 없습니다.** 그래서 자체 검출 전환 기준을 1.5초에서 1초로 앞당겼습니다.

한계. 보행자 행동 비율과 가속도는 가정값이고, 검출 실패(박스가 안 나옴)는 넣지 않았습니다. 1:10 모형 시나리오 S9(주행 중 공유기 끄기)에서 같은 지표를 다시 재서 확인합니다.

5.3 파이 자체 검출은 왜 필요한가

	칼만 예측 모드	파이 자체 검출
볼 수 있는 것	끊기기 전에 추적하던 물체만	새로 나타난 물체 포함 전부
계산량	물체당 수십 µs	프레임당 수십~100 ms (추정, 실측 필요)
믿을 수 있는 시간	약 1초 (5.2절)	제한 없음
약점	급정지·방향 전환, 새 물체	입력이 작아(320 px) 먼 물체를 놓침
역할	끊긴 직후 공백 메우기	1초 이후 감시 유지 + '감시 저하' 경보

끊긴 뒤 골목에서 새로 뛰어나온 사람은 예측으로는 절대 볼 수 없습니다. 반대로 자체 검출은 켜는 데 시간이 걸리고 정확도가 낮습니다. **두 방법이 서로의 빈틈을 메우므로 둘 다 필요합니다.** 선택안으로, 평소에도 파이가 초당 2~3장씩 자체 검출을

돌려 노트북 결과와 교차 확인하면 파이 모델의 실제 성능을 상시 측정할 수 있습니다.

파이 5에서 YOLO11n NCNN(입력 320)의 속도는 공개 벤치마크 기억으로 대략 초당 5~15장 수준으로 추정합니다. 2주차에 직접 재서 합격선(5 FPS 이상)을 확인합니다.

5.4 그 밖의 문제

문제	원인	해결	확인 방법
무선 지연이 큼	공용 와이파이 혼잡	5 GHz 전용 공유기, JPEG 품질·해상도 조절, 0.3초 넘은 결과 버림	2주차 왕복 지연 측정
먼 거리 오차	레일 간격이 몇 픽셀뿐	칼만 평활, 거리별 측정 잡음, 먼 쪽만 잘라 검출	눈금 검증 표
경보 깜빡임	검출 흔들림	올릴 땐 2프레임, 내릴 땐 1초 유지	S4 등급 변화 횟수
레일이 가려짐	사람·차가 레일을 덮음	직선 식으로 이어 그림, 바닥 평면·사람 키 보조	S8 거리 오차
파이 발열	자체 검출 시 부하	액티브 쿨러, 온도 로그	10분 연속 실행 온도
조명 변화	역광·어두움	카메라 모듈 3 HDR, 노출 고정, 학습 데이터에 조명 3종	S7 검출률

6. 모델 학습 계획

6.1 전략

처음부터 학습하지 않습니다. 이미 사람과 차를 아는 COCO 사전학습 YOLO11n을 그대로 써서 3주차에 시험장 사진으로 성능을 먼저 잽니다. 모형 피규어는 실제 사람과 질감·비율이 달라 놓칠 수 있으므로, **거리 구간별 재현율이 90% 미만이면** 우리 사진으로 미세조정(전이학습)합니다.

6.2 단계

단계	할 일	도구
① 수집	시험장 영상에서 1초에 1~2장 추출, 300~500장. 조명 3종, 거리 1~5 m, 가림 포함, 물체 없는 사진 10%	휴대폰·카메라 모듈 3, OpenCV
② 라벨링	person, car, bicycle 박스 (COCO 번호 유지), 2명이 교차 확인	Roboflow, CVAT, Label Studio 중 하나
③ 나누기	학습 70% · 검증 20% · 시험 10%, **영상 단위로** 나눠 비슷한 앞뒤 프레임이 섞이지 않게	스크립트
④ 학습	yolo11n.pt에서 시작, epochs 100, imgsz 640, 기본 증강	Ultralytics, Google Colab 무료 GPU
⑤ 검증	mAP50, 거리 구간별 재현율, 노트북·파이 FPS	`yolo val`
⑥ 변환	노트북용 OpenVINO, 파이용 NCNN 320 (INT8 양자화는 선택)	`yolo export`

6.3 명령 예시

```
# tram.yaml : path, train/val/test 폴더, names {0: person, 1: bicycle, 2: car}
yolo train model=yolo11n.pt data=tram.yaml epochs=100 imgsz=640
yolo val   model=runs/detect/train/weights/best.pt data=tram.yaml split=test
yolo export model=best.pt format=openvino          # 노트북
yolo export model=best.pt format=ncnn imgsz=320    # 라즈베리파이
```

6.4 합격선 (제안)

- 모형 3 m 안 사람(피규어) 재현율 95% 이상
- 노트북 OpenVINO 30 FPS 이상, 파이 NCNN 320 5 FPS 이상
- 학습에 쓰지 않은 시험 사진으로만 보고

6.5 라이선스 주의

Ultralytics YOLO는 AGPL-3.0입니다. 동아리 저장소를 공개로 두면 교육·대회 용도에는 문제가 없지만, 상업화하려면 별도 라이선스가 필요합니다. RailSem19 데이터셋은 이용 조건(연구용)을 확인한 뒤 씁니다.

7. 활용할 수 있는 코드와 예제

7.1 직접 만든 예제 (project-files/code_examples)

파일	하는 일	상태
00_make_sample_video.py	정답을 아는 합성 영상 + 정답 CSV	기존 예제
01_opencv_basics.py	영상 읽기, 엣지, 직선, 그리기	기존 예제
02_yolo_detect_track.py	YOLO 검출 + ByteTrack ID 추적	기존 예제
03_rail_detect.py	테이프 선로 검출, 궤간 픽셀 폭, 위험 통로	기존 예제
04_distance.py	궤간·사람 키로 앞/옆 거리	기존 예제
05_risk_level.py	속도·TTC·필요 감속도, 4단계, 깜빡임 방지	기존 예제
06_pipeline.py	PC 전체 데모 + 이벤트 로그	기존 예제
07_kalman_tracker.py	바닥 평면 칼만 추적기, 거리별 측정 잡음, 예측만 하기	**새로 추가, PC 실행 확인**
08_dropout_sim.py	연결 끊김 3,000회 시뮬레이션 (5.2절 표)	**새로 추가, PC 실행 확인**
09_laptop_server.py	노트북: 사진 받아 검출·선로 결과만 돌려줌	**새로 추가, PC 시험**
10_pi_client.py	파이: 촬영·전송, 추적·판정·경보, 끊김 대응	**새로 추가, PC 시험**

PC 한 대로 분담 구조를 시험하는 방법:

```
python 00_make_sample_video.py
python 09_laptop_server.py --detector color                                  # 창 1
python 10_pi_client.py --source sample.mp4 --local color --drop-at 3 --drop-for 2.5   # 창 2
```

파이에서는 `--server tcp://<노트북IP>:5555 --source picam --local yolo --local-model yolo11n_ncnn_model` 로 실행합니다. 파이 실기 시험은 보드와 카메라가 준비된 뒤 진행합니다.

7.2 가져다 쓰는 오픈소스

라이선스는 기억에 기반한 것이므로 사용 전에 각 저장소에서 다시 확인합니다.

이름	쓰는 곳	라이선스
Ultralytics YOLO (문서: 학습, track, export, Raspberry Pi 가이드)	검출 · 학습 · 변환	AGPL-3.0
picamera2 (공식 예제 포함)	파이 카메라 촬영	BSD-2
pyzmq (ZeroMQ), imagezmq	사진 · 결과 전송	BSD / MIT

이름	쓰는 곳	라이선스
OpenCV (카메라 보정 튜토리얼)	보정 · 선로 · 그리기	Apache-2.0
filterpy	칼만 필터 참고 구현 (예제 07은 numpy만 사용)	MIT
SORT (abewley/sort)	추적 구조 원본 참고	GPL-3.0
gpiozero	부저 · LED	BSD
RailSafeNet	레일 분할 + 궤간 거리 구역 아이디어	MIT

8. 추후 진행 과정

주차	목표	주요 할 일	완료 기준
2주 (이번 주)	준비 · 관문	부품 발주, 파이↔노트북 왕복 지연 측정, 예제 07~10을 팀원 PC에서 실행	왕복 0.3초 이내
3~4주	기초 구현	카메라 보정, 하드보드 시험장, 선로·거리·등급, 사전학습 YOLO 성능 측정	PC 데모, 눈금 검증 표
5~7주	온디바이스 통합	파이에서 추적·판단·경보, 연결 끊김 대응, 필요하면 미세조정	모형 실시간 경보 (MVP)
8~10주	시험 · 발표	시나리오 S1~S9 각 10회, 분담 vs 파이 단독 비교, 결과 정리	최종 발표

시험 시나리오

번호	상황	기대 결과	지표
S1	선로 위 정지 피규어, 주행체 접근	주의 → 경고 → 위험 순서로 상승	리드타임, TTC 오차
S2	선로 옆(통로 밖)을 나란히 이동	경고 이상 없음	오경보율
S3	옆에서 선로로 가로지름	진입 전에 경고	놓침률, 리드타임
S4	들어왔다가 다시 나감	상승 후 해제, 깜빡임 없음	등급 변화 횟수
S5	여러 개 (피규어 2 + 차 1)	가장 위험한 것 기준 경보	ID 유지율
S6	곡선 구간	곡선 통로 정상 생성	선로 검출 성공률
S7	어두운 조명 · 역광	성능 저하 폭 측정	검출률 변화
S8	레일 일부 가림	보조 거리 방법으로 전환	거리 오차
S9	주행 중 공유기 끄기	0.3초 안에 예측 모드, 1초에 자체 검출	모드 전환 시간, 끊긴 동안 놓침

Plan B. 무선 지연이 0.3초를 넘으면 주행 시험만 유선(이더넷)으로 연결합니다. 그래도 안 되면 파이 단독 검출(NCNN 320)로 전체를 돌리고, 분담 구조와의 성능 차이를 결과로 보고합니다.

부록. 숫자의 출처

숫자	출처	확인 필요
부품 가격	기억에 기반한 추정	발주 전 판매가
카메라 모듈 3 화각 66°, fx 985 px	사양 기억 + 계산	보정으로 실측
제동거리 94 m (50 km/h), 상용 제동 1.2 m/s²	예시값	실제 차량 자료
칼만 오차 0.99 → 0.51 m, 11 µs	예제 07 실행 결과 (PC)	파이에서 재실행
시뮬레이션 표	예제 08 실행 결과 (가정한 보행자 행동)	모형 시나리오 S9
지연 예산, 파이 YOLO 속도	추정	2주차 실측

숫자	출처	확인 필요
논문 서지 정보, 라이선스	기억 기반 정리	원문 · 저장소 확인